Christian DiPietrantonio
ME 5311: Computational Methods to Viscous Flows
April 27, 2026

Computer Assignment 03: Calculating Turbulent Boundary Layer Development

1. Description of the Problem

The objective of this project is to use the code developed in Project 02 for the laminar boundary layer to calculate the development of a turbulent boundary layer using the algebraic model described below. In the inner region of the boundary layer, the Reichardt model is used, and the eddy viscocity is given by

$$\nu_t = \kappa\nu \left[\frac{y u_\tau}{\nu} - y_a^+ \tanh\left(\frac{y u_\tau}{\nu y_a^+} \right) \right] \tag{1}$$

Where $y_a^+ = 9.7$. In the outer region, the Clauser model is used, which assumes that the eddy viscocity reaches a constant value of $(\nu_t)_{\max} = c_1 U_e \delta(x)$, where $\delta(x)$ is taken to be the location where $u = 0.994 U_\infty$. Whenever ν_t calculated from Reichardt's model exceeds $(\nu_t)_{\max}$, the Clauser model is used and the derivative of ν_t is set to zero. Model constants $\kappa = 0.41$ and $c_1 = 0.001$ are used, and the friction velocity is found by taking the numerical derivative of u at the wall. The same dimensional properties from Project 02 are used, with $y_{\max} = 50\delta$ and $N = 250$. The objectives are to: Calculate $\delta(x)$ and $c_f(x)$ for each x location and confirm that the $\delta(x)$ grows like $\delta(x) = 0.375x(Re_x)^{-1/5}$; obtain the apparent origion of the turbulent boundary layer $x_{0,\text{turb}}$; confirm the boundary layer follows the "law of the wall"; determine the sensitivity of the model to κ, c_1, and s; discuss the necesary changes in grid resolution and stretching factor when Re is increased by a factor of 10; and discuss implementation issues.

2. Description of the Numerical Method

The numerical method used is nearly identical to Project 02, the Crank-Nicolson algorithm is used to march in the x-direction, a sinh-based stretched grid is used in η, a fourth-order central difference scheme is used for the discretization of η (except at the first interior points, where a second-order discretization is used), and banded linear solvers are used for both u^* and v^* at each x-step. However, for this project ν^* and $\frac{\partial \nu^*}{\partial \eta}$ are no longer constants ($\nu^* = 1, \frac{\partial \nu^*}{\partial \eta} = 0$), instead they are computed from the turbulence model at each x-step. Nevertheless, the coefficients A_j and B_j retain the same formula as in Project 02:

$$A_j = \frac{v_j^*}{J_j^*} - \frac{C}{J_j^{*2}} \frac{\partial \nu^*}{\partial \eta}\bigg|_j + \frac{C\,\nu_j^*\,J_{\eta,j}^*}{J_j^{*3}}, \qquad B_j = \frac{-C\,\nu_j^*}{J_j^{*2}} \tag{2}$$

The friction velocity is nondimensionalized as u_τ^*/U_∞. From the definition $u_\tau^2 = \nu\,(\partial u/\partial y)|_w$ and the chain rule, the follwoing relationship is obtained:

$$u_\tau^{*2} = \frac{1}{Re_\delta\,J_0^*} \frac{\partial u^*}{\partial \eta}\bigg|_0 \tag{3}$$

Where $Re_\delta = U_\infty \delta/\nu_\infty$ and the wall gradient is computed via a 2nd-order forward difference

stencil:

$$\left.\frac{\partial u^*}{\partial \eta}\right|_0 = \frac{4u_1^* - u_2^*}{2\Delta\eta} \tag{4}$$

The wall coordinate is expressed as $y^+ = y^* u_\tau^* Re_\delta$, and the nondimensionalized effective viscotity as $\nu_{eff}^* = 1 + \nu_t/\nu_\infty$. The Reichardt and Clauser models can then be nondimensionalized by dividing by ν_∞:

$$\frac{\nu_t}{\nu_\infty} = \kappa \left[y^+ - y_a^+ \tanh\left(\frac{y^+}{y_a^+}\right)\right], \qquad \frac{\nu_{t,\max}}{\nu_\infty} = c_1 \, Re_\delta \, y_{0.994}^* \tag{5}$$

Where $y_{0.994}^*$ is the y^* location where $u^* = 0.994$, found by linear interpolation of the generated data. Therefore, the nondimensional effective viscocity can be expressed in the Reichardt and Clauser regions as

$$\nu_{eff}^* = 1 + \kappa \left[y^+ - y_a^+ \tanh\left(\frac{y^+}{y_a^+}\right)\right], \qquad \nu_{eff}^* = 1 + c_1 \, Re_\delta \, y_{0.994}^* \tag{6}$$

Finally, the derivtive required for A_j can be derived using the chain rule for both regions:

$$\frac{\partial \nu_{eff}^*}{\partial \eta} = \begin{cases} \kappa \, u_\tau^* \, Re_\delta \, J^* \left[1 - \operatorname{sech}^2\left(\frac{y^+}{y_a^+}\right)\right], & \text{Reichardt region} \\ 0, & \text{Clauser region (constant } \nu_t) \end{cases} \tag{7}$$

At the wall ($y^+ = 0$), $\operatorname{sech}^2(0) = 1$, so $d\nu^*/d\eta = 0$. Furthermore, the skin friction coefficient can be expressed in terms of the nondimensional friction velocity as $c_f = 2(u_\tau^*)^2$. The models and logic described above can now be implemented into the program written for Project 02, the result of which is displayed in Appendix A.

3. Presentation of Results

The solution is marched from $x_0^* = 20.74$ to $x_{\max}^* = 60.74$ using $N_x = 20{,}000$ steps. At the first grid point, $y_1^+ \approx 0.07$, confirming the point is within the viscous sublayer.

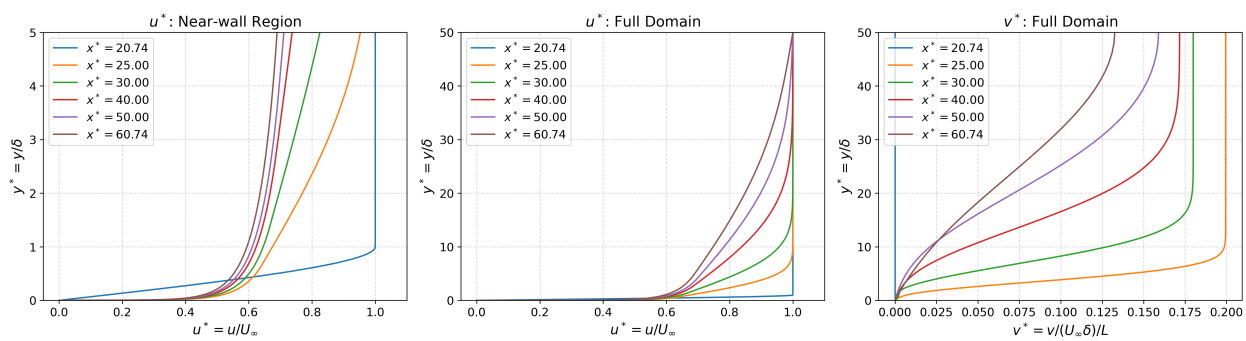

Figure 1: Velocity profiles at downstream locations. Left: u^* vs. y^* near-wall zoom showing the initial profile. Middle: u^* vs. y^* full domain showing boundary layer growth from $\sim 1\delta$ to $\sim 48\delta$. Right: v^* vs. y^* showing the wall-normal profiles.

The apparent origin of the turbulent boundary layer $x_{0,\text{turb}} = 21.3$ m was obtained by linearizing the provided growth correlation. Raising $\delta = C(x - x_{0,\text{turb}})^{4/5}$ to the power of 5/4 gives $\delta^{5/4} = C^{5/4}(x - x_{0,\text{turb}})$, which is linear in x and equals zero at $x = x_{0,\text{turb}}$. Following this, a linear regression was used on the downstream data (filtered with $H < 1.4$ to omit laminar results) to obtain $x_{0,\text{turb}}$. Figure 2 below shows agreement between the numerical

$\delta(x)$ and the corelation the turbulent region.

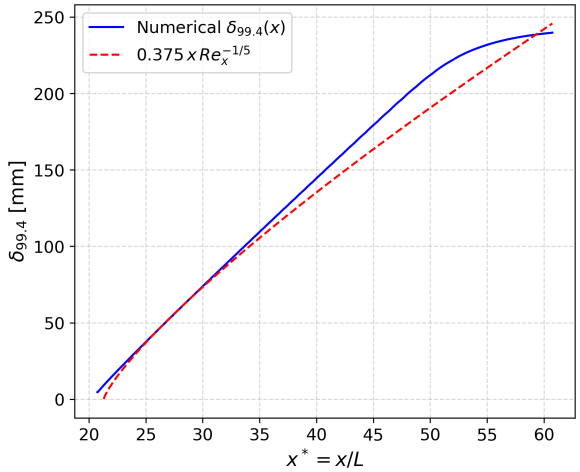

Figure 2: Boundary layer growth: numerical $\delta_{99.4}(x)$ vs. empirical correlation.

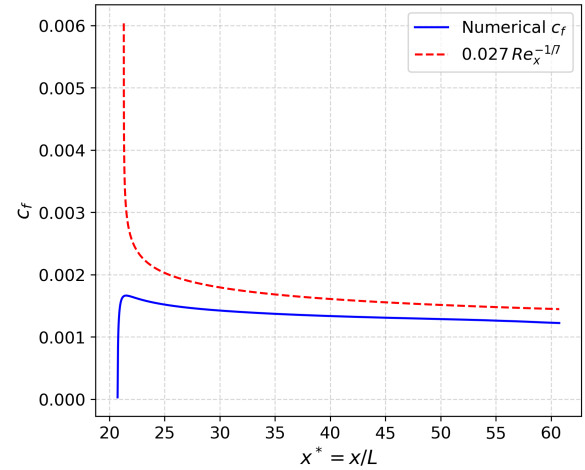

Figure 3: Skin friction coefficient: numerical $c_f(x)$ vs. empirical correlation.

Table 1: Boundary layer quantities at selected streamwise locations.

x^*	$\delta_{99.4}/\delta$	δ^*/δ	θ/δ	H	c_f	u_τ^*
20.74	0.936	0.375	0.139	2.692	2.998×10^{-5}	3.872×10^{-3}
25.00	7.480	1.106	0.808	1.368	1.510×10^{-3}	2.757×10^{-2}
30.00	14.87	2.063	1.541	1.339	1.424×10^{-3}	$2.668e \times 10^{-2}$
40.00	28.91	3.829	2.916	1.313	1.336×10^{-3}	2.584×10^{-2}
50.00	42.50	5.477	4.210	1.301	1.287×10^{-3}	2.537×10^{-2}
60.74	47.96	7.056	5.385	1.310	1.224×10^{-3}	2.474×10^{-2}

The shape factor drops rapidly from $H \approx 2.69$ (laminar) to $H \approx 1.3$ (turbulent) between $x^* = 20.74$ and $x^* = 25$, confirming a rapid trransition from the initial cubic laminar profile. The downstream values of $H \approx 1.3$ are in agreement with the theoretical shape factor for a turbulent boundary layer on a smooth flat plate.

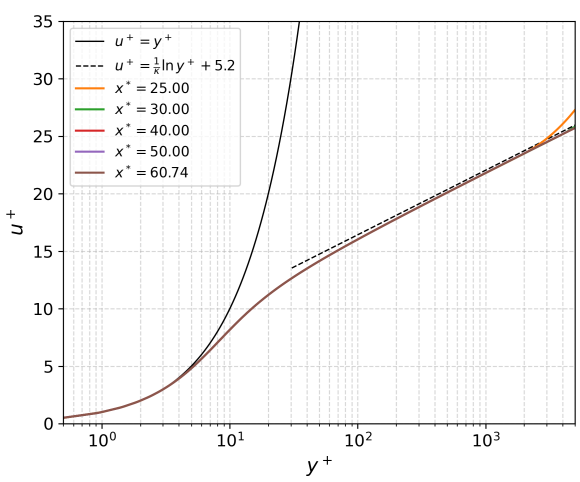

Figure 4: Law of the wall: u^+ vs. y^+ compared with $u^+ = y^+$ and $u^+ = (1/\kappa)\ln y^+ + 5.2$.

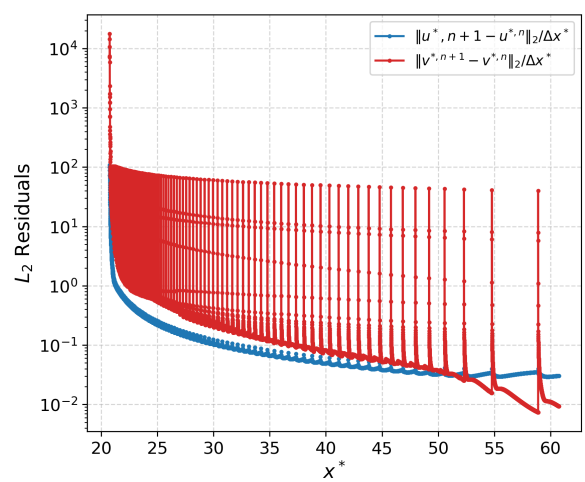

Figure 5: L_2 residuals (normalized by Δx^*) vs. x^*.

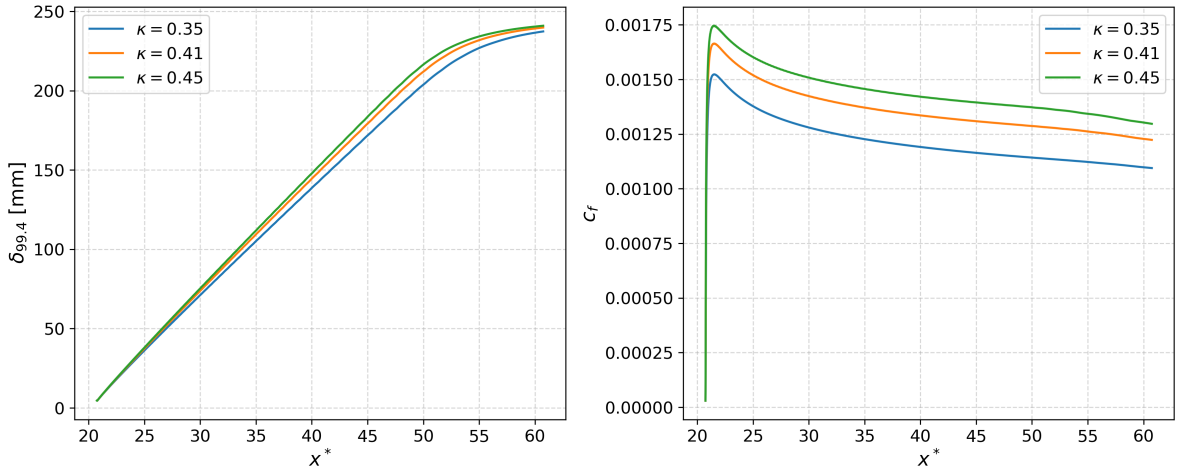

Figure 6: Sensitivity to κ ($c_1 = 0.001$, $s = 10$ fixed).

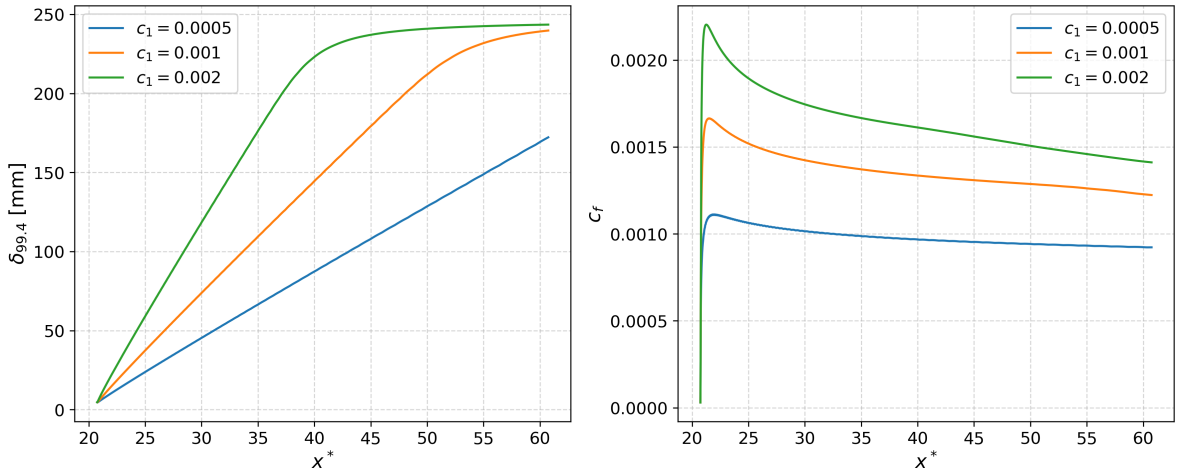

Figure 7: Sensitivity to c_1 ($\kappa = 0.41$, $s = 10$ fixed).

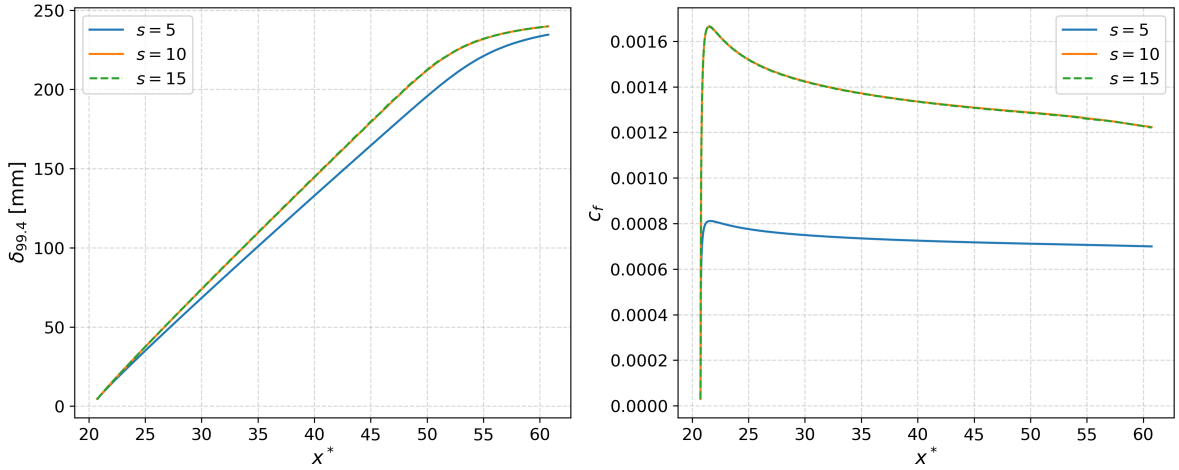

Figure 8: Sensitivity to Stretching Factor s ($\kappa = 0.41$, $c_1 = 0.001$ fixed).

4. Discussion of Results

4.1. General description

Figure 1 shows that the initial cubic laminar profile at $x^* = 27.74$ is confined to $y^* \leq 1$, while by $x^* = 60.74$, $\delta_{99.4} \approx 48\delta$, approaching $y_{\max} = 50\delta$. The v^* profiles confirm the satisfaction of continuity. Inside the boundary layer, $\frac{\partial u^*}{\partial x*} < 0$ (the boundary layer is growing), so $\frac{\partial v^*}{\partial y*} > 0$ (fluid is displaced away from the wall). Outside the boundary layer, $u^* = 1$, so both derivaives are zero and v^* assymptotes to a constant. This constant decreases with downstream distance because $\frac{\partial u^*}{\partial x*}$ decreases in magnitude (as the boundary layer growth rate slows).

Agreement between the simulated c_f and the empirical correlation can be observed in Figure 3. The initial disagreement is expected because the correlation assumes a fully turbulent boudary layer growing from zero thickness at $x_{0,\text{turb}}$, which predicts a large gradient at the wall for a small boundary layer. Whereas, at $x_{0,\text{turb}}$ the numerical solution is still relaxing from the laminar cubic profile, which has a smaller gradient in the wall in comparison. Figure 4 confirms that the numerical solution cloesly agreess with the law of the wall, with a slight deviation in the outer region for $x^* = 25$, as again, the solution relaxes from the initial condition.

Figures 6–8 show the sensitivity to the three model/grid parameters. At $x^* = 40.74$, varying κ by 1% produces roughly a 0.24% to 0.28% change in δ and a 0.66% to 0.74% change in c_f, while varying c_1 by 1% produces roughly a 0.5% to 0.8% change in δ and a 0.21% to 0.55% change in c_f. Therefore, c_f is more sensitive to κ and $\delta_{99.4}$ is more sensitive to c_1. This makes sense intuitively, as κ controls inner region viscocity/mixing (Reichardt model) where the wall gradient is determined, and c_1 controls outer region viscocity/mixing (Clauser model) where increased mixing spreads the velocity transition over a wider distance, increasing δ. For the stretching factor, the results for $s = 10$ and $s = 15$ are nearly identical, demonstrating grid convergance. However, $s = 5$ differs significantly due to insufficient near-wall resoultion and because its first point falls within the buffer layer ($y^+ \approx 25.7$).

Increasing the Reynolds number by a factor of 10 would create a larger velocity gradient at the wall, increasing u_τ and effectively reducing the thickness of the viscous sublayer. This would require a larger stretching factor s (and as a result, possibly more grid points N to maintain fidelity in the outer layer) to resolve the sharper gradient and keep y_1^+ within the viscous sublayer.

An implementation challenge was the $\text{sech}^2(y^+/y_a^+)$ term in $d\nu^*/d\eta$, which resulted in overflow errors for large y^+/y_a^+. Since $\text{sech}^2(x) \to 0$ for large x. Line 160 in Appendix A clamps the argument to 50 ($\text{sech}^2(50) \approx 10^{-43}$), which allows the program to procede witout errors. Another implementation issue encountered was sever fluctuationsin the v^* residual, which are addressed in the Accuracy and Stability section below.

4.2. Accuracy and stability

The u^* residual in Figure 5 decreases as x^* increases (as to be expected with the Crank-Nicolson scheme), which indicates convergence of the solution. The large initial residual is a result of the solutions rapit adjustment from the laminar initial condition. The v^* residual exhibits oscillations that are suspected to result from the transition between the Reichardt and Clauser regions. BUT HOW. Since v^* is derived from $\partial u/\partial x$, these discontinuities are amplified. Nevertheless, physically intuitive results are still obtained for v^* at each step.

A. Copy of Program Listing

```python
# Christian DiPietrantonio
# ME 5311: Computational Methods to Viscous Flows
# Computer Assignment 03
# 04/10/2026

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.linalg import solve_banded

# ------------------------------------------------------------------
# 1. SET PARAMETERS
# ------------------------------------------------------------------

def set_parameters():
    """Define all physical constants, grid parameters, and nondimensional quantities."""

    params = {}

    # --- physical properties ---
    params["nu_inf"]        = 1.5e-6
    params["U_inf"]         = 30.0
    params["L"]             = 1.0
    params["delta"]         = 0.005

    # --- grid parameters ---
    params["N"]             = 250   # number of grid points in eta (y-direction)
    params["y_max"]         = 50 * params["delta"]  # final y
    params["s"]             = 10    # stretching factor
    params["deta"]          = 1.0 / (params["N"] - 1)  # step size in eta

    # --- turbulence model constants ---
    params["kappa"]         = 0.41
    params["c1"]            = 0.001
    params["ya_plus"]       = 9.7

    # --- nondimensional quantities ---
    params["Re_L"]          = params["U_inf"] * params["L"] / params["nu_inf"]
    params["Re_delta"]      = params["U_inf"] * params["delta"] / params["nu_inf"]
    params["C"]             = (1.0 / params["Re_L"]) * (params["L"] / params["delta"])**2
        # non-dimensional coefficient

    # --- streamwise domain ---
    params["x0"]            = (params["U_inf"] * params["delta"]**2) / (params["nu_inf"]
        * 4.91**2) # initial x from Blasius relation
    params["x0_star"]       = params["x0"] / params["L"]  # non-dimensional initial x, x0
        *
    params["x_max"]         = params["x0"] + 40 * params["L"] # final x
    params["x_max_star"]    = params["x_max"] / params["L"]  # non-dimensional final x,
        x_max*
    params["Nx"]            = 20000    # number of steps in x*
```

```python
    params["dx_star"]       = (params["x_max_star"] - params["x0_star"]) / params["Nx"] # step size in x*

    # -- output locations ---
    params["x_output"]      = [params["x0_star"], 25, 30, 40, 50, params["x_max_star"]]

    return params

# ------------------------------------------------------------------
# 2. COMPUTE GRID
# ------------------------------------------------------------------

def compute_grid(params):
    """Compute the stretched grid, the Jacobian and its derivative."""

    N         = params["N"]
    y_max     = params["y_max"]
    s         = params["s"]
    delta     = params["delta"]
    eta       = np.linspace(0.0, 1.0, N)
    y_phys    = y_max * np.sinh(s * eta) / np.sinh(s)
    y_star    = y_phys / delta
    J_star    = (y_max * s * np.cosh(s * eta)) / (delta * np.sinh(s))
    J_star_eta = (y_max * s**2 * np.sinh(s * eta)) / (delta * np.sinh(s))

    return eta, y_phys, y_star, J_star, J_star_eta

# ------------------------------------------------------------------
# 3. COMPUTE INITIAL CONDITION
# ------------------------------------------------------------------

def compute_initial_condition(params, y_star):
    """Compute the initial velocity profile at x0."""

    N = params["N"]
    u_star_0 = np.ones(N)

    for j in range(N):
        y_j = y_star[j]

        if y_j <= 1.0:
            u_star_0[j] = y_j * (1.5 - 0.5 * y_j**2)
        else:
            u_star_0[j] = 1.0

    return u_star_0

# ------------------------------------------------------------------
# 4.  COMPUTE nu* and dnu*/deta
# ------------------------------------------------------------------

def compute_nu(params, u_star, J_star):
    """Compute the non-dimensional kinematic viscosity nu* and its derivative with respect to eta."""
```

```python

    N         = params["N"]
    deta      = params["deta"]
    kappa     = params["kappa"]
    c1        = params["c1"]
    ya_plus   = params["ya_plus"]
    Re_delta  = params["Re_delta"]

    # ----- Compute Friction Velocity -----
    du_deta_wall = (4.0 * u_star[1] - u_star[2]) / (2.0 * deta)

    u_tau_star_sq = du_deta_wall / (Re_delta * J_star[0])

    """
    if u_tau_star_sq < 0:
        u_tau_star_sq = 1e-15
    """

    u_tau_star = np.sqrt(u_tau_star_sq)

    # ----- Compute y+ at each grid point -----
    y_max        = params["y_max"]
    s            = params["s"]
    delta        = params["delta"]
    eta_arr      = np.linspace(0.0, 1.0, N)
    y_star_arr   = (y_max / delta) * np.sinh(s * eta_arr) / np.sinh(s)
    y_plus       = y_star_arr * u_tau_star * Re_delta

    # ----- Compute Reichardt Eddy Viscosity -----
    nu_t_star    = kappa * (y_plus - ya_plus * np.tanh(y_plus / ya_plus))

    # ----- Compute Clauser Eddy Viscosity -----
    idx_994 = np.searchsorted(u_star, 0.994)     # first index where u* > 0.994

    if idx_994 <=0:
        y_star_994 = y_star_arr[0]               # edge case for if velocity is already >= 0.994
    elif idx_994 >= N:
        y_star_994 = y_star_arr[-1]              # edge case for if velocity never reaches 0.994
    else:
        u_below = u_star[idx_994 - 1]            # u* just below 0.994
        u_above = u_star[idx_994]                # u* just above 0.994
        y_below = y_star_arr[idx_994 - 1]        # corresponding y*
        y_above = y_star_arr[idx_994]            # corresponding y*
        if abs(u_above - u_below) > 1e-15:
            frac = (0.994 - u_below)/(u_above - u_below)
            y_star_994 = y_below + frac * (y_above - y_below) # linearly interpolate for y* at u* = 0.994
        else:
            y_star_994 = y_below

    nu_t_star_max = c1 * y_star_994 * Re_delta

```

```python
    # ----- Apply Clauser Condition
    clauser_mask = nu_t_star > nu_t_star_max # true where Clauser is active
    nu_t_star[clauser_mask] = nu_t_star_max

    # ----- Nondimensional Effective Viscosity -----
    nu_star_eff = 1 + nu_t_star

    # ----- Compute Derivative of Effective Viscosity -----
    arg = np.minimum(y_plus / ya_plus, 50)     # clamp to prevent overflow errors
    sech_sq = 1.0 / np.cosh(arg)**2
    dnu_star_eff_deta = kappa * u_tau_star * Re_delta * J_star * (1 - sech_sq)

    dnu_star_eff_deta[clauser_mask] = 0.0     # apply Clauser condition to derivative

    #dnu_star_eff_deta[0] = 0.0               # not needed

    return nu_star_eff, dnu_star_eff_deta, u_tau_star, y_star_994

# ------------------------------------------------------------------
# 5. COMPUTE COEFFICIENTS A_j and B_j
# ------------------------------------------------------------------

def compute_coefficients(params, v_star, J_star, J_star_eta, nu_star_eff, dnu_star_eff_deta):
    """Compute the coefficients A_j and B_j for the discretized system."""

    C = params["C"]
    A_j = (v_star / J_star) - ((C / J_star**2) * dnu_star_eff_deta) + ((C * nu_star_eff * J_star_eta) / J_star**3)
    B_j = -C * nu_star_eff / J_star**2

    return A_j, B_j

# ------------------------------------------------------------------
# 6. COMPUTE v* FROM CONTINUITY
# ------------------------------------------------------------------

def compute_v_star(u_star_new, u_star_old, u_star_prev, params, J_star):
    """Compute v* from the transformed/non-dimensional continuity equation
       using a 4th-order approximation for dv*/deta and a 2nd-order approximation
       for du*/dx* for the interior domain (j = 2 to N-3), and a 4th-order approximation
       for dv*/deta and a 1st-order approximation for du*/dx* near the boundary/at
       the initial step in x (i = 1, j = 1 and j = N-2)."""

    N         = params["N"]
    dx_star   = params["dx_star"]
    deta      = params["deta"]
    l_diag        = 2 # number of lower diagonals in banded system
    u_diag        = 1 # number of upper diagonals in banded system

    # ---------- Compute du*/dx* at each grid point ---------------
    if u_star_prev is None:
        du_dx_star = (u_star_new - u_star_old) / dx_star
    else:
```

```python
        du_dx_star = (3.0 * u_star_new - 4.0 * u_star_old + u_star_prev) / (2.0 * dx_star)

    # ---------- Compute midpoint RHS values -----------------------
    f_mid = np.zeros(N)
    for j in range(1, N):
        J_mid             = 0.5 * (J_star[j] + J_star[j-1])
        du_dx_star_mid    = 0.5 * (du_dx_star[j] + du_dx_star[j-1])
        f_mid[j]          = -J_mid * du_dx_star_mid

    # ---------- Set up banded system -----------------------------
    # Number of unknowns = N-1, v*_0 = 0 (Wall BC)
    # Unknowns: x[k] = v*_{k+1} for k = 0 to N-2
    n = N - 1 # number of unknowns (v_star[1] to v_star[N-1])
    c = 1.0 / (24.0 * deta) # stencil coefficient

    # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 1 upper diagonal
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((l_diag + u_diag + 1, n))
    rhs_v_star = np.zeros(n)

    # Row 0: Midpoint Rule for v*_1
    ab[u_diag, 0] = 1.0 / deta  # [row=0, col=0] -> ab[1 + 0 - 0, 0] = ab[1, 0]
    rhs_v_star[0] = f_mid[1]

    # Rows 1 to N-3 (corresponding to v*_2 to v*_{N-2}): 4th order stencil
    for j in range(2, N-1):
        r = j - 1                           # row index in banded system

        if j - 2 >= 1:
            ab[u_diag + r - (r-2), r-2] = c  # map value two columns behind main diagonal (2nd lower diagonal)
                                            # ab[3, r-2]: v*_{j-2}
        # if j = 2, v*_0 = 0, so no contribution

        ab[u_diag + r - (r-1), r-1] = -27 * c # map value one column behind main diagonal (1st lower diagonal)
                                            # ab[2, r-1]: v*_{j-1}

        ab[u_diag, r]             = 27 * c    # map value on main diagonal
                                            # ab[1, r]: v*_j

        ab[u_diag + r - (r+1), r+1] = -c     # map value one column ahead of main diagonal (1st upper diagonal)
                                            # ab[0, r+1]: v*_{j+1}

        rhs_v_star[r] = f_mid[j]

    # Row N-2: Midpoint Rule for v*_{N-1}
    r = N - 2
    ab[u_diag + r - (r-1), r-1] = -1.0 / deta # map value one column behind main diagonal (1st lower diagonal)
                                            # ab[2, r-1]: v*_{N-2}

    ab[u_diag + r - r, r]     = 1.0 / deta    # map value on main diagonal
```

```
                                                # ab[1, r]: v*_{N-1}

    rhs_v_star[r] = f_mid[N-1]

    # ---------- Solve banded system -------------------------------
    x = solve_banded((l_diag, u_diag), ab, rhs_v_star)

    # ---------- Assemble full v_star array ------------------------
    v_star = np.zeros(N)
    v_star[0] = 0.0        # wall BC
    v_star[1:] = x         # interior and freestream points

    return v_star

# ------------------------------------------------------------------
# 7.  BUILD LINEAR SYSTEM FOR MOMENTUM EQUATION
# ------------------------------------------------------------------

def build_system(params, u_old, A_j, B_j):
    """Build the banded matrix and RHS vector for the linear system resulting from the discretization
      of the momentum equation."""

    N         = params["N"]
    dx_star   = params["dx_star"]
    deta      = params["deta"]
    l_diag    = 2 # number of lower diagonals in banded system
    u_diag    = 2 # number of upper diagonals in banded system

    # Number of unknowns = N-2, u*_0 = 0 (Wall BC), u*_{N-1} = 1 (Freestream BC)
    # Unknowns: x[k] = u*_{k+1} for k = 0 to N-3
    n         = N - 2 # number of unknowns (u_star[1] to u_star[N-2])

    # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 2 upper diagonals
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((l_diag + u_diag + 1, n))
    rhs = np.zeros(n)

    # Row 0: j=1 (near wall, 2nd order)
    j = 1; k = 0
    psi_1        = A_j[j] / (4.0 * deta)
    psi_2        = B_j[j] / (2.0 * deta**2)
    ab[2, k]     = (u_old[j] / dx_star) - 2 * psi_2 # [row=0, col=0] -> ab[2 + 0 - 0, 0] = ab[2, 0]
    ab[1, k + 1] = psi_1 + psi_2                    # [row=0, col=1] -> ab[2 + 0 - 1, 1] = ab[1, 1]
    rhs[k]       = ((u_old[j]**2 / dx_star) + 2.0 * psi_2 * u_old[j]) + (-psi_1 - psi_2) * u_old[j+1]

    # Rows 1 to N-4 (corresponding to u*_2 to u*_{N-3}): 4th order stencil
    for j in range(2, N-2):
        k = j - 1  # row index in banded system
        phi_1    = A_j[j] / (24.0 * deta)
        phi_2    = B_j[j] / (24.0 * deta**2)
```

```
        c_jm2    = phi_1 - phi_2                        # coefficient for u^{*,i+1}_{j-2}
        c_jm1    = -8.0 * phi_1 + 16.0 * phi_2          # coefficient for u^{*,i+1}_{j-1}
        c_j      = u_old[j] / dx_star - 30.0 * phi_2    # coefficient for u^{*,i+1}_j
        c_jp1    = 8.0 * phi_1 + 16.0 * phi_2           # coefficient for u^{*,i+1}_{j+1}
        c_jp2    = -phi_1 - phi_2                       # coefficient for u^{*,i+1}_{j+2}
        r_jm2    = -phi_1 + phi_2                       # coefficient for u^{*,i}_{j-2}
        r_jm1    = 8.0 * phi_1 - 16.0 * phi_2           # coefficient for u^{*,i}_{j-1}
        r_j      = u_old[j] / dx_star + 30.0 * phi_2    # coefficient for u^{*,i}_j
        r_jp1    = -8.0 * phi_1 - 16.0 * phi_2          # coefficient for u^{*,i}_{j+1}
        r_jp2    = phi_1 + phi_2                        # coefficient for u^{*,i}_{j+2}

        if j - 2 >= 1:
            ab[u_diag + k - (k-2), k-2]   = c_jm2 # map value two columns behind main diagonal (2nd lower diagonal)
                                                  # ab[4, k-2]: u*_{j-2}

        ab[u_diag + k - (k-1), k-1]      = c_jm1 # map value one column behind main diagonal (1st lower diagonal)
                                                  # ab[3, k-1]: u*_{j-1}

        ab[u_diag, k]                    = c_j   # map value on main diagonal
                                                  # ab[2, k]: u*_j

        ab[u_diag + k - (k+1), k+1]      = c_jp1 # map value one column ahead of main diagonal (1st upper diagonal)
                                                  # ab[1, k+1]: u*_{j+1}

        if j + 2 <= N - 2:
            ab[u_diag + k - (k+2), k+2]   = c_jp2 # map value two columns ahead of main diagonal (2nd upper diagonal)
                                                  # ab[0, k+2]: u*_{j+2}

        rhs_val = r_j * u_old[j]

        if j - 2 >= 1:
            rhs_val += r_jm2 * u_old[j-2]    # contribution from u^{*,i}_{j-2}
        rhs_val += r_jm1 * u_old[j-1]        # contribution from u^{*,i}_{j-1}
        rhs_val += r_jp1 * u_old[j+1]        # contribution from u^{*,i}_{j+1}
        if j + 2 <= N - 2:
            rhs_val += r_jp2 * u_old[j+2]    # contribution from u^{*,i}_{j+2}
        else:
            rhs_val += r_jp2 * 1.0           # contribution from u^{*,i}_{N-1} = 1.0 (Freestream BC)
            rhs_val -= c_jp2 * 1.0           # contribution from u^{*,i+1}_{N-1} = 1.0 (Freestream BC)
        rhs[k] = rhs_val

    # Row N-3: j=N-2 (near freestream, 2nd order)
    j = N - 2; k = j - 1
    psi_1              = A_j[j] / (4.0 * deta)
    psi_2              = B_j[j] / (2.0 * deta**2)
    ab[3, k-1] = -psi_1 + psi_2                   # [row=N-3, col=N-4] -> ab[2 + N-3 - N-4, N-4] = ab[3, N-4]
```

```python
    ab[2, k]  = u_old[j] / dx_star - 2.0 * psi_2  # [row=N-3, col=N-3] -> ab[2 + N-3 - N-3, N-3] = ab[2, N-3]
    # include contributions from u^{*,i+1}_{N-1} and u^{*,i}_{N-1} = 1.0 (Freestream BC) in RHS
    rhs[k]    = (psi_1 - psi_2) * u_old[j-1] + (u_old[j]**2 / dx_star) + (2.0 * psi_2 * u_old[j]) + (-psi_1 - psi_2) * 1.0 - (psi_1 + psi_2) * 1.0

    return ab, rhs, l_diag, u_diag

# ------------------------------------------------------------------
# 8.  STEP IN X*
# ------------------------------------------------------------------

def step_in_x(params, u_star_0, J_star, J_star_eta):
    """March from x0_star to x_max_star"""

    N        = params["N"]
    Nx       = params["Nx"]
    dx_star  = params["dx_star"]
    x0_star  = params["x0_star"]

    u_star = u_star_0.copy()
    v_star = np.zeros(N)
    u_star_prev = None                          # No previous step initially -> use 1st order du/dx in compute_v_star

    stride = max(1, Nx // 20000)                 # store solution every stride steps

    store_indices = set(range(0, Nx + 1, stride)) # indices/step numbers at which to store solution

    for x_out in params["x_output"]:
        idx = int(round((x_out - x0_star) / dx_star)) # find step number corresponding to x_out
        idx = max(0, min(idx, Nx))               # ensure idx is within bounds
        store_indices.add(idx)                   # add idx to set store_indices (if not already present)

    store_indices = sorted(store_indices)        # sort indices for ordered storage
    n_store = len(store_indices)

    x_vals = np.zeros(n_store)                   # initialize array to store solution x-values
    u_store = np.zeros((n_store, N))             # initialize array to store solution profiles
    delta_store = np.zeros(n_store)              # initialize array to store BL thicknesses
    cf_store = np.zeros(n_store)                 # initialize array to store Cf coefficients
    u_tau_star_store = np.zeros(n_store)         # initialize array to store nondimensional friction velocities
    v_store = np.zeros((n_store, N))             # initialize array to store v* profiles

    # initialize residual storage
```

```python
    x_res_list = []
    u_res_list = []
    v_res_list = []
    v_star_prev_res = None

    store_map = {idx: pos for pos, idx in enumerate(store_indices)} # map from step number to storage index

    # --- Store Initial Conditions ---
    if 0 in store_map:
        pos = store_map[0]
        x_vals[pos] = x0_star            # store initial condition at x0_star if it's in the output list
        u_store[pos, :] = u_star.copy()
        v_store[pos, :] = v_star.copy()

        # compute initial turbulent quantities
        nu_star_eff_0, dnu_star_eff_0, u_tau_star_0, y_star_994_0 = compute_nu(params, u_star, J_star)
        delta_store[pos]    = y_star_994_0       # BL Thickness
        cf_store[pos]       = 2 * u_tau_star_0**2 # Skin Friction Coefficient
        u_tau_star_store[pos] = u_tau_star_0     # Nondimensional Friction Velocity

    # --- march in x ---
    for n_step in range(Nx):
        # compute effective viscosity from current velocity profile
        nu_star_eff_n, dnu_star_eff_n, u_tau_star_n, y_star_994_n = compute_nu(params, u_star, J_star)

        # compute coefficients A_j and B_j
        A_j, B_j = compute_coefficients(params, v_star, J_star, J_star_eta, nu_star_eff_n, dnu_star_eff_n)

        # build and solve the linear system for u* at the next x-step
        ab, rhs_vec, l_diag, u_diag = build_system(params, u_star, A_j, B_j)
        u_interior = solve_banded((l_diag, u_diag), ab, rhs_vec)

        # assemble the new velocity profile
        u_star_new          = np.zeros(N)
        u_star_new[0]       = 0.0         # wall BC
        u_star_new[1:N-1]   = u_interior  # interior points
        u_star_new[N-1]     = 1.0         # freestream BC

        # Compute v_star using continuity equation
        v_star = compute_v_star(u_star_new, u_star, u_star_prev, params, J_star)

        # update for next step
        u_star_prev = u_star.copy() # store current u_star before updating for next iteration
        u_star = u_star_new.copy() # update u_star for next iteration

        # calculate residuals at every step, ensuring there is a previous point to compare with
        if u_star_prev is not None and v_star_prev_res is not None:
```

```python
            u_res_list.append(np.linalg.norm(u_star - u_star_prev, ord=2) / dx_star)
            v_res_list.append(np.linalg.norm(v_star - v_star_prev_res, ord=2) / dx_star)
            x_res_list.append(x0_star + (n_step + 1) * dx_star)
        v_star_prev_res = v_star.copy()

        step = n_step + 1
        if step in store_map:
            pos = store_map[step]               # find storage index for current step
            x_vals[pos] = x0_star + step * dx_star # compute and store x-value for current step
            u_store[pos, :] = u_star.copy()     # store solution profile for current step
            v_store[pos, :] = v_star.copy()     # store v* profile for current step

            # compute turbulent quantities at this step
            nu_star_eff_s, dnu_star_eff_s, u_tau_star_s, y_star_994_s = compute_nu(params, u_star, J_star)
            delta_store[pos]    = y_star_994_s        # BL Thickness
            cf_store[pos]       = 2 * u_tau_star_s**2 # Skin Friction Coefficient
            u_tau_star_store[pos] = u_tau_star_s      # Nondimensional Friction Velocity

        # if step % stride == 0:
        if (n_step + 1) % 1000 == 0:
            # print(f" Step {step}/{Nx}, x* = {x0_star + (step) * dx_star:.2f}")
            print(f" Step {n_step+1}/{Nx}, x* = {x0_star + (n_step+1) * dx_star:.2f}, u*_tau = {u_tau_star_n:.6f}, delta_99/delta = {y_star_994_n:.4f}")

    return x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, stride, x_res_list, u_res_list, v_res_list

# ------------------------------------------------------------------
# 9.  COMPUTE BLASIUS SOLUTION
# ------------------------------------------------------------------

def compute_blasius(eta_max=10.0, n_points = 10000):
    """Solve Blasius via RK4."""

    h = eta_max / n_points       # step size for RK4

    eta_B = np.zeros(n_points + 1)      # values for eta
    f_arr = np.zeros(n_points + 1)      # stream function at each eta
    F_arr = np.zeros(n_points + 1)      # velocity profile u/U_inf = u*
    H_arr = np.zeros(n_points + 1)      # d2f/deta2

    f_arr[0] = 0.0                      # f(0) = 0 at the wall
    F_arr[0] = 0.0                      # F(0) = 0 at the wall
    H_arr[0] = 0.332                    # H(0) that results in F(n->infty) = 1

    def rhs(f_val, F_val, H_val):

        df = F_val                      # df/deta = F
        dF = H_val                      # dF/deta = H
        dH = -0.5 * f_val * H_val       # dH/deta = -(f/2)*H

        return df, dF, dH
```

```python

    # Implement RK4 numerical method
    for i in range(n_points):
        eta_B[i] = i * h
        fi, Fi, Hi = f_arr[i], F_arr[i], H_arr[i]

        k1f, k1F, k1H = rhs(fi, Fi, Hi)
        k2f, k2F, k2H = rhs(fi + 0.5*h*k1f, Fi + 0.5*h*k1F, Hi + 0.5*h*k1H)
        k3f, k3F, k3H = rhs(fi + 0.5*h*k2f, Fi + 0.5*h*k2F, Hi + 0.5*h*k2H)
        k4f, k4F, k4H = rhs(fi + h*k3f, Fi + h*k3F, Hi + h*k3H)

        f_arr[i+1] = fi + (h/6.0)*(k1f + 2*k2f + 2*k3f + k4f)
        F_arr[i+1] = Fi + (h/6.0)*(k1F + 2*k2F + 2*k3F + k4F)
        H_arr[i+1] = Hi + (h/6.0)*(k1H + 2*k2H + 2*k3H + k4H)

    eta_B[-1] = n_points * h

    return eta_B, F_arr, f_arr # eta_B, u*/U_infty = F, stream function f

# ---------------------------------------------------------------------
# 10. COMPUTE THICKNESSES AND SHAPE FACTOR
# ---------------------------------------------------------------------

def compute_thicknesses(u_star, J_star, deta):
    """Compute displacement thickness, momentum thickness, and shape factor"""
    integrand_disp  = (1.0 - u_star) * J_star
    integrand_mom   = u_star * (1.0 - u_star) * J_star

    delta_star      = np.trapezoid(integrand_disp, dx=deta)  #integrate for displacement thickness using trapezoidal rule
    theta           = np.trapezoid(integrand_mom, dx=deta)   #integrate for momentum thickness using trapezoidal rule

    H               = delta_star / theta if theta > 1e-15 else 0.0

    return delta_star, theta, H

# ---------------------------------------------------------------------
# 11. POST-PROCESSING
# ---------------------------------------------------------------------

def post_process(params, x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, eta, y_phys, y_star, J_star, stride, x_res_list, u_res_list, v_res_list):
    N         = params["N"]
    deta      = params["deta"]
    nu        = params["nu_inf"]
    U_inf     = params["U_inf"]
    L         = params["L"]
    delta     = params["delta"]
    Re_delta  = params["Re_delta"]
    kappa     = params["kappa"]

    print("Computing Blasius solution...")
```

```python
    eta_B, F_B, f_B = compute_blasius(eta_max=10.0, n_points = 10000)

    output_indices = []
    for x_out in params["x_output"]:
        idx = np.argmin(np.abs(x_vals - x_out))     #take difference between desired x and stored x and return index of lowest entry
        output_indices.append(idx)

    colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple', 'tab:brown']

    # ----- Plot 01: u* and v* vs. y* -----
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for i, idx in enumerate(output_indices):
        x_now = x_vals[idx]
        u_num = u_store[idx, :]
        v_num = v_store[idx, :]

        # Left panel: u* near-wall zoom
        axes[0].plot(u_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5, label=fr'$x^*={x_now:.2f}$')

        # Middle panel: u* full domain
        axes[1].plot(u_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5, label=fr'$x^*={x_now:.2f}$')

        # Right panel: v* full domain
        axes[2].plot(v_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5, label=fr'$x^*={x_now:.2f}$')

    # Left panel formatting
    axes[0].set_xlabel(r'$u^* = u / U_\infty$', fontsize=14)
    axes[0].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
    axes[0].set_xlim([-0.05, 1.1])
    axes[0].set_ylim([0.0, 5.0])
    axes[0].set_title(r'$u^*$: Near-wall Region', fontsize=15)
    axes[0].tick_params(labelsize=12)
    axes[0].grid(True, linestyle='--', alpha=0.5)
    axes[0].legend(fontsize=12, loc='upper left')

    # Middle panel formatting
    axes[1].set_xlabel(r'$u^* = u / U_\infty$', fontsize=14)
    axes[1].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
    axes[1].set_xlim([-0.05, 1.1])
    axes[1].set_ylim([0.0, y_star[-1]])
    #axes[1].set_ylim([0.0, 60])
    axes[1].set_title(r'$u^*$: Full Domain', fontsize=15)
    axes[1].tick_params(labelsize=12)
    axes[1].grid(True, linestyle='--', alpha=0.5)
    axes[1].legend(fontsize=12, loc='upper left')

    # Right panel formatting
    axes[2].set_xlabel(r'$v^* = v / (U_\infty \delta) / L$', fontsize=14)
```

```python
    axes[2].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
    axes[2].set_ylim([0.0, y_star[-1]])
    axes[2].set_title(r'$v^*$: Full Domain', fontsize=15)
    axes[2].tick_params(labelsize=12)
    axes[2].grid(True, linestyle='--', alpha=0.5)
    axes[2].legend(fontsize=12, loc='upper left')

    plt.tight_layout()
    plt.savefig('velocity_profiles.png', dpi=300, bbox_inches='tight')
    print('    ...Saved velocity_profiles.png')

    # ----- Plot 02: Boundary Layer Growth -----
    delta_994 = delta_store * delta # meters
    x_phys = x_vals * L

    n_data = len(x_vals)
    #i_start = n_data // 3  # skip initial laminar profile

    # calculate shape factors at each output location
    H_store = np.zeros(len(x_vals))
    for i in range(len(x_vals)):
        _, _, H_store[i] = compute_thicknesses(u_store[i, :], J_star, deta)

    H_threshold = 1.4   # threshold for turbulent shape factor (laminar ~ 2.59, turbulent ~ 1.3)
    turbulent_indices = np.where(H_store < H_threshold)[0] # record indices where H < threshold

    if len(turbulent_indices) > 0:
        i_start = turbulent_indices[0]
    else:
        i_start = 0   # if no turbulent indices, just start at begining of the data

    turb_mask = (delta_994[i_start:] > 0) & (x_phys[i_start:] > 0) # create boolean mask for turbulent indices (taking only values with positive x and delta values)

    x_fit           = x_phys[i_start:][turb_mask]              # apply masks to filtered arrays
    delta_994_fit   = delta_994[i_start:][turb_mask]
    delta_994_54    = delta_994_fit**(5.0/4.0)

    if len(x_fit) > 2:
        coeffs    = np.polyfit(x_fit, delta_994_54, 1)
        slope_fit = coeffs[0]
        x0_turb   = -coeffs[1] / slope_fit if abs(slope_fit) > 1e-15 else 0.0
    else:
        x0_turb   = x_phys[0]   # if not enough points for curve fit, just set x0_turb to initial x location

    x0_turb_star = x0_turb / L

    print(f'\n   Turbulent BL apparent origin:')
    print(f'     x0_turb = {x0_turb:.2f} m (x0_turb* = {x0_turb_star:.2f})')

```

```python
    # compute the empirical correlation delta(x) for comparison
    x_corr    = x_phys - x0_turb       # distance from apparent origin
    x_corr_pos = np.maximum(x_corr, 1e-15) # avoid negative/zero values
    Re_x_corr = U_inf * x_corr_pos / nu
    delta_corr = 0.375 * x_corr_pos * Re_x_corr**(-1.0/5.0)

    plt.figure(figsize=(6, 5))
    plt.plot(x_vals, delta_994 * 1000, 'b-', linewidth=1.5, label=r'Numerical $\delta_{99.4}(x)$')
    plot_mask = x_corr > 0       # boolean mask so that only turbulent values are plotted for numerical correlations
    plt.plot(x_vals[plot_mask], delta_corr[plot_mask] * 1000, 'r--', linewidth=1.5, label=r'$0.375\, x\, Re_x^{-1/5}$')
    plt.xlabel(r'$x^* = x/L$', fontsize=14)
    plt.ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=12, loc='best')
    plt.tight_layout()
    plt.savefig('BL_growth.png', dpi=300, bbox_inches='tight')
    print('    ...Saved BL_growth.png')

    # ----- Plot 03: Skin Friction Coefficient -----
    # Emperical turbulent flat-plate correlation: cf = 0.059 * Rex^{-1/5} or 0.027 * Rex^{-1/7}
    cf_corr          = np.zeros_like(x_vals)
    #cf_corr[plot_mask] = 0.059 * Re_x_corr[plot_mask]**(-1.0/5.0) # only use correlation for turbulent data
    cf_corr[plot_mask] = 0.027 * Re_x_corr[plot_mask]**(-1.0/7.0) # only use correlation for turbulent data

    plt.figure(figsize=(6, 5))
    plt.plot(x_vals, cf_store, 'b-', linewidth=1.5, label=r'Numerical $c_f$')
    #plt.plot(x_vals[plot_mask], cf_corr[plot_mask], 'r--', linewidth=1.5, label=r'$0.059\,Re_x^{-1/5}$')
    plt.plot(x_vals[plot_mask], cf_corr[plot_mask], 'r--', linewidth=1.5, label=r'$0.027\,Re_x^{-1/7}$')
    plt.xlabel(r'$x^* = x/L$', fontsize=14)
    plt.ylabel(r'$c_f$', fontsize=14)
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=12, loc='best')
    plt.tight_layout()
    plt.savefig('SkinFrictionCoeff.png', dpi=300, bbox_inches='tight')
    print('    ...Saved SkinFrictionCoeff.png')

    # ----- Plot 04: The Law of the Wall -----
    #plot a selected downstream locations

    plt.figure(figsize=(6, 5))

    y_plus_theor = np.logspace(-1, 4, 500)
    u_plus_visc = y_plus_theor                              # viscous sublayer u+ = y+
    u_plus_log  = (1.0 / kappa) * np.log(y_plus_theor) + 5.2 # log law region
```

```python

    plt.semilogx(y_plus_theor, u_plus_visc, 'k-', linewidth=1, label=r'$u^+ = y^+$')
    plt.semilogx(y_plus_theor[y_plus_theor > 30], u_plus_log[y_plus_theor > 30], 'k--',
        linewidth=1, label=r'$u^+ = \frac{1}{\kappa}\ln y^+ + 5.2$')

    for i, idx in enumerate(output_indices):
        if i == 0:        # skip initial condition
            continue

        x_now     = x_vals[idx]
        u_num     = u_store[idx, :]
        u_tau_star = u_tau_star_store[idx]

        if u_tau_star < 1e-15:       # protect against dividing by zero
            continue

        y_plus_local = y_star * Re_delta * u_tau_star
        u_plus_local = u_num / u_tau_star

        plt.semilogx(y_plus_local[1:], u_plus_local[1:], '-', color=colors[i % len(colors)
            ], linewidth=1.5, label=fr'$x^*={x_now:.2f}$') #avoid y*=0

    plt.xlabel(r'$y^+$', fontsize=14)
    plt.ylabel(r'$u^+$', fontsize=14)
    plt.xlim([0.5, 5e3])
    plt.ylim([0, 35])
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5, which='both')
    plt.legend(fontsize=10, loc='upper left')
    plt.tight_layout()
    plt.savefig('LawOfTheWall.png', dpi=300, bbox_inches='tight')
    print('    ...Saved LawOfTheWall.png')

    #Plot 05: L2 Residual
    plt.figure(figsize=(6, 5))
    plt.plot(x_res_list, u_res_list, 'o-', color='tab:blue', linewidth=1.5, markersize=2,
         label=r'$\|u^*,{n+1} - u^{*,n}\|_2 / \Delta x^*$')
    plt.plot(x_res_list, v_res_list, 'o-', color='tab:red', linewidth=1.5, markersize=2,
        label=r'$\|v^{*,n+1} - v^{*,n}\|_2 / \Delta x^*$')
    plt.yscale('log')
    plt.xlabel(r'$x^*$', fontsize=14)
    plt.ylabel('$L_2$ Residuals', fontsize=14)
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=10, loc='best')
    plt.tight_layout()
    plt.savefig('residuals_vs_x.png', dpi=300, bbox_inches='tight')
    print("...Saved residuals_vs_x.png")

    #Table: Thicknesses
    print("\n" + "="*120)
    print(f"{'x*':>10} {'x [m]':>10} {'delta_99/d':>12} {'d*/d':>12} {'th/d':>12} "
         f"{'H':>12} {'cf':>12} {'u_tau*':>12}")
    print("="*120)
```

```python
    for i, idx in enumerate(output_indices):
        x_now     = x_vals[idx]
        x_dim     = x_now * L
        u_num     = u_store[idx, :]
        delta_star, theta, H = compute_thicknesses(u_num, J_star, deta)
        print(f"{x_now:10.2f} {x_dim:10.2f} {delta_store[idx]:12.4f} {delta_star:12.6f} {theta:12.6f} {H:12.4f} {cf_store[idx]:12.4e} {u_tau_star_store[idx]:12.4e}")
    print("="*120 )

    return x0_turb, x0_turb_star

# ------------------------------------------------------------------
# 12. SENSITIVITY STUDY
# ------------------------------------------------------------------

def run_sensitivity(base_params, u_star_0, J_star, J_star_eta, y_star):
    """
    Run the solver with different values of cappa and c1 to asses sensitivity of the turbulence model.
    """

    kappa_vals = [0.35, 0.41, 0.45]      # test values for von Karman constant
    c1_vals    = [0.0005, 0.001, 0.002]   # test values for clauser constant
    s_vals     = [5, 10, 15]              # test values for stretching factor

    results_kappa = {}
    results_c1    = {}
    results_s     = {}

    # --- Sensitivity to kappa (hold c1 = 0.001 fixed) ---
    print('\n' + '=' * 120)
    print('SENSITIVITY STUDY: Varying kappa (c1 = 0.001 fixed)')
    print('=' * 120)
    for kap in kappa_vals:
        params = base_params.copy()
        params["kappa"] = kap
        params["c1"]   = 0.001
        print(f'\n    Running kappa = {kap}...')
        x_v, u_s, v_s, d_s, cf_s, ut_s, stride, _, _, _ = step_in_x(params, u_star_0.copy(), J_star, J_star_eta)
        results_kappa[kap] = (x_v, d_s, cf_s)

    # --- Sensitivity to c1 (hold kappa = 0.41 fixed) ---
    print('\n' + '=' * 120)
    print("SENSITIVITY STUDY: Varying c1 (kappa = 0.41 fixed)")
    print('=' * 120)
    for c1v in c1_vals:
        params = base_params.copy()
        params["kappa"]    = 0.41
        params["c1"]       = c1v
        print(f'\n    Running c1 = {c1v}...')
        x_v, u_s, v_s, d_s, cf_s, ut_s, stride, _, _, _ = step_in_x(params, u_star_0.copy(), J_star, J_star_eta)
        results_c1[c1v] = (x_v, d_s, cf_s)
```

```python

    # --- Sensitivity to s (hold kappa = 0.41 and c1 = 0.001 fixed) ---
    print('\n' + '=' * 120)
    print("SENSITIVITY STUDY: Varying s (kappa = 0.41, c1 = 0.001 fixed)")
    print('=' * 120)
    for s_val in s_vals:
        params = base_params.copy()
        params["kappa"]   = 0.41
        params["c1"]      = 0.001
        params["s"]      = s_val
        print(f'\n    Running s = {s_val}...')
        _, _, y_star_s, J_star_s, J_star_eta_s = compute_grid(params)
        u_star_0_s = compute_initial_condition(params, y_star_s)
        x_v, u_s, v_s, d_s, cf_s, ut_s, stride, _, _, _ = step_in_x(params, u_star_0_s,
            J_star_s, J_star_eta_s)
        results_s[s_val] = (x_v, d_s, cf_s)

    # --- Plot sensitivity to kappa ---
    delta_ref = base_params["delta"]
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    for kap in kappa_vals:
        x_v, d_s, cf_s = results_kappa[kap]
        axes[0].plot(x_v, d_s * delta_ref * 1000, linewidth=1.5, label=fr'$\kappa={kap}$')
        axes[1].plot(x_v, cf_s, linewidth=1.5, label=fr'$\kappa={kap}$')

    axes[0].set_xlabel(r'$x^*$', fontsize=14)
    axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
    axes[0].legend(fontsize=12)
    axes[0].grid(True, linestyle='--', alpha=0.5)
    axes[0].tick_params(labelsize=12)
    axes[1].set_xlabel(r'$x^*$', fontsize=14)
    axes[1].set_ylabel(r'$c_f$', fontsize=14)
    axes[1].legend(fontsize=12)
    axes[1].grid(True, linestyle='--', alpha=0.5)
    axes[1].tick_params(labelsize=12)
    #plt.suptitle(r'Sensitivity to $\kappa$ ($c_1 = 0.001$)', fontsize=14)
    plt.tight_layout()
    plt.savefig('Kappa_Sensitivity.png', dpi=300, bbox_inches='tight')
    print('\n    ...Saved Kappa_Sensitivity.png')

    # --- Plot sensitivity to c1 ---
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    for c1v in c1_vals:
        x_v, d_s, cf_s = results_c1[c1v]
        axes[0].plot(x_v, d_s * delta_ref * 1000, linewidth=1.5, label=fr'$c_1={c1v}$')
        axes[1].plot(x_v, cf_s, linewidth=1.5, label=fr'$c_1={c1v}$')

    axes[0].set_xlabel(r'$x^*$', fontsize=14)
    axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
    axes[0].legend(fontsize=12)
    axes[0].grid(True, linestyle='--', alpha=0.5)
    axes[0].tick_params(labelsize=12)
```

```python
    axes[1].set_xlabel(r'$x^*$', fontsize=14)
    axes[1].set_ylabel(r'$c_f$', fontsize=14)
    axes[1].legend(fontsize=12)
    axes[1].grid(True, linestyle='--', alpha=0.5)
    axes[1].tick_params(labelsize=12)
    #plt.suptitle(r'Sensitivity to $c_1$ ($\kappa = 0.41$)', fontsize=14)
    plt.tight_layout()
    plt.savefig('c1_Sensitivity.png', dpi=300, bbox_inches='tight')
    print('\n    ...Saved c1_Sensitivity.png')

    # --- Plot sensitivity to s ---
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    s_styles = ['-', '-', '--']
    for style, s_val in enumerate(s_vals):
        x_v, d_s, cf_s = results_s[s_val]
        axes[0].plot(x_v, d_s * delta_ref * 1000, linestyle=s_styles[style], linewidth=1.5, label=fr'$s={s_val}$')
        axes[1].plot(x_v, cf_s, linestyle=s_styles[style], linewidth=1.5, label=fr'$s={s_val}$')

    axes[0].set_xlabel(r'$x^*$', fontsize=14)
    axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
    axes[0].legend(fontsize=12)
    axes[0].grid(True, linestyle='--', alpha=0.5)
    axes[0].tick_params(labelsize=12)
    axes[1].set_xlabel(r'$x^*$', fontsize=14)
    axes[1].set_ylabel(r'$c_f$', fontsize=14)
    axes[1].legend(fontsize=12)
    axes[1].grid(True, linestyle='--', alpha=0.5)
    axes[1].tick_params(labelsize=12)
    #plt.suptitle(r'Sensitivity to $s$ ($\kappa = 0.41$, $c_1 = 0.001$)', fontsize=14)
    plt.tight_layout()
    plt.savefig('s_Sensitivity.png', dpi=300, bbox_inches='tight')
    print('\n    ...Saved s_Sensitivity.png')

# ---------------------------------------------------------------------
# 12. MAIN DRIVER
# ---------------------------------------------------------------------
def main():
    print("=" * 120)
    print( "    ME 5311 - Project 03: Turbulent Boundary Layer Solver")
    print( "    Reichardt (inner region) + Clauser Model (outer region)")
    print("=" * 120)

    # --- Step 1: Set Parameters ---
    print("\n[1] Setting parameters...")
    params = set_parameters()
    print(f"    Re_L     = {params['Re_L']:.2e}")
    print(f"    Re_delta = {params['Re_delta']:.2e}")
    print(f"    x0*      = {params['x0_star']:.2f}")
    print(f"    N        = {params['N']}")
    print(f"    y_max    = {params['y_max']} m  ({params['y_max']/params['delta']:.0f} delta)")
    print(f"    kappa    = {params['kappa']}")
```

```
    print(f"    c1        = {params['c1']}")
    print(f"    ya+       = {params['ya_plus']}")
    print(f"    Nx        = {params['Nx']},   dx* = {params['dx_star']:6f}")

    # --- Step 2: Compute stretched grid ---
    print("\n[2] Computing stretched grid...")
    eta, y_phys, y_star, J_star, J_star_eta = compute_grid(params)
    print(f"    dy_min = {y_phys[1]-y_phys[0]:.2e} m, dy_max = {y_phys[-1]-y_phys[-2]:.2e} m")
    print(f"    y*_min = {y_star[1]:.4f},  y*_max = {y_star[-1]:.4f}")

    #check if first point is within viscous sublayer
    u_star_0     = compute_initial_condition(params, y_star)
    du_deta_0    = (4.0 * u_star_0[1] - u_star_0[2]) / (2.0 * params["deta"])
    #u_tau_init   = np.sqrt(max(du_deta_0 / (params['Re_delta'] * J_star[0])), 1e-15)
    u_tau_init   = np.sqrt(du_deta_0 / (params["Re_delta"] * J_star[0]))
    y_plus_init  = y_star[1] * params["Re_delta"] * u_tau_init
    print(f"    Initial u_tau* ~ {u_tau_init:.6f}, y+_1 ~ {y_plus_init:.3f}")

    # --- Step 3: Set Initial Conditions ---
    print("\n[3] Setting initial conditions...")
    #u_star_0 = compute_initial_condition(params, y_star)

    # --- Step 4: March in x ---
    print("\n[4] Marching in x...")
    x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, stride, x_res_list, u_res_list, v_res_list = step_in_x(params, u_star_0, J_star, J_star_eta)

    # --- Step 5: Post Process ---
    print("\n[5] Post-processing...")
    x0_turb, x0_turb_star = post_process(params, x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, eta, y_phys, y_star, J_star, stride, x_res_list, u_res_list, v_res_list)

    # --- Step 6: Sensitivity Study ---
    print("\n[6] Running sensitivity study ...")
    run_sensitivity(params, u_star_0, J_star, J_star_eta, y_star)

    print("\n" + "=" * 120)
    print("\nDone!")
    print("=" * 120)

if __name__ == "__main__":
    main()
```

A. Other Notes

B. Description of the Problem

The objective of this project is to use the code developed in Project 02 for the laminar boundary layer to calculate the development of a turbulent boundary layer using the algebraic model described below. In the inner region of the boundary layer, the Reichardt model is used, and the eddy viscocity is given by

$$\nu_t = \kappa\nu \left[\frac{yu_\tau}{\nu} - y_a^+ \tanh\left(\frac{yu_\tau}{\nu y_a^+} \right) \right] \tag{8}$$

Where $y_a^+ = 9.7$. In the outer region, the Clauser model is used, which assumes that the eddy viscocity reaches a constant value of $(\nu_t)_{\max} = c_1 U_e \delta(x)$, where $\delta(x)$ is taken to be the location where $u = 0.994U_\infty$. Whenever ν_t calculated from Reichardt's model exceeds $(\nu_t)_{\max}$, the Clauser model is used and the derivative of ν_t is set to zero. Model constants $\kappa = 0.41$ and $c_1 = 0.001$ are used, and the friction velocity is found by taking the numerical derivative of u at the wall. For consistant results with Project 02, the expressions for ν_t and their spatial derivatives must be nondimensionalized. Further objectives of this project are to:

1. Calculate the turbulent boundary layer flow for the same dimensional properties used in the laminar case. However, this time using a value of $y_{\max} = 50\delta$ and 250 points in the y direction. Calculate $\delta(x)$ and $c_f(x)$ for each x location and confirm that the boundary layer grows like

$$\delta(x) = 0.375x(Re_x)^{-1/5} \tag{9}$$

2. Adjust the results to account for the transition from the assumed initial laminar velocity profile by obtaining a value for x_0, the apparent origin of the turbulent boundary layer.

3. Show that the turbulent boundary layer follows the "law of the wall."

4. Determine the sensitivity of the turbulence model to the choices of κ and c_1.

5. Determine what changes are necessary in the grid resolution and stretching factor to obtain a converged solution when the Reynolds number is increased by a factor of 10.

6. Discuss the problems encountered with the implementation of this model.

The apparent origin of the turbulent boundary layer $x_{0,\text{turb}}$ was found to be at 21.3 meters, and was obtained by linearizing the correlation: $\delta(x) = 0.375x(Re_x)^{-1/5}$. Since $\delta = C(x - x_{0,\text{turb}})^{4/5}$, raising both sides to the power of 5/4 results in $\delta^{5/4} = C^{5/4}(x - x_{0,\text{turb}})$, which is linear in x. Since when $x = x_{0,\text{turb}}$, $\delta(x) = 0$, and $\delta^{5/4} = 0$, a linear regression was used on the downstream data to solve for the $x_{0,\text{turb}}$ at which $\delta^{5/4} = 0$. It should be noted that the data used for the linear regression was filtered using a shape factor threshold $H < 1.4$, to ensure data from the initial laminar profile was omitted. In Figure 2 below, agreement between the numerical $\delta(x)$ and the aforementioned corelation can be observed over the turbulent region. The L2 residual for v^*, however, seems to show sever oscilations. It is suspected that this is a result of several factors. Primarily, since v^* is calculated from the continuity equation at each

step, it is derived from the derivative of the velocity profile, which can amplify changes that may be smoothed out by the Crank-Nicolson scheme. Second, these differences bay result from the transition between the Reichardt and Clauser regions of the boundary layer, which suggests that these instabilities are a property of the chosen turbulence models, and not an implementation error. Nevertheless, physically intuitive results are still obtained for u^* and v^* at each step, the accuracy of which has been validated against theoretical corelations.

The v^* residual exhibits oscillations that are believed to result from the discrete transition between Reichardt and Clauser regions: as the boundary layer grows, the crossover point passes through successive grid cells, causing step changes in $d\nu^*/d\eta$ (which jumps from its Reichardt value to zero at each grid point that transitions to the Clauser region). Since v^* is derived from $\partial u^*/\partial x^*$ through continuity, these discontinuities are amplified in the v^* computation. Despite these residual oscillations, the v^* profiles themselves are physically consistent at every station, and all results have been validated against theoretical correlations.

Finally, several problems with implementation of the model were encountered. First, the equation for the derivative of the effective viscocity contains a $\text{sech}^2(y^+/y_a^+)$ term. For large y^+, the argument y^+/y_a^+ becomes very large, and computing $\cosh y^+/y_a^+$ results in overfow errors. However, $\text{sech}^2(x) \to 0$ for large x, so line 160 in Appendix A resolves this by clamping the argument to a maximum of 50. Since $\text{sech}^2(50) \approx 1.5 \times 10^{-43}$ is effectively zero, this logic allows the program to proceed without encountering overflow errors. Another implementation issue encountered was sever fluctuationsin the v^* residual, which are addressed in the Accuracy and Stability section below.